

Aplicação de Algoritmos Genéticos ao TSP

Métodos de otimização e aplicações — PPGComp

Bruno S. Maion, Lucca A. Neres, Karolayne Dambrosio
Programa de Pós-Graduação em Ciência da Computação
CCET - Unioeste

8 de dezembro de 2025

Sumário

1	Introdução	1
2	Materiais e métodos	2
2.1	Ambiente Computacional	2
2.2	Algoritmos Genéticos Implementados	2
2.2.1	Parâmetros AG	2
2.2.2	AG's Implementados	2
2.3	Instâncias de Testes	3
2.4	Métricas de Desempenho	3
2.5	Clusterização	3
2.6	Particularidades de implementação	4
3	Resultados	5
3.1	Berlin52	5
3.2	CH150	6
3.3	RD400	7
3.4	Resultados gerais	9
4	Conclusões	10

1 Introdução

O Problema do Caixeiro Viajante (TSP) é um desafio clássico de otimização combinatória, classificado como NP-difícil, cujo objetivo é encontrar o ciclo de menor custo que visita um conjunto de cidades exatamente uma vez ([Arenales et al., 2015](#)). Devido à sua complexidade para grandes instâncias, meta-heurísticas como os Algoritmos Genéticos (AGs) são utilizadas para encontrar soluções de boa qualidade em tempo satisfatório. Este trabalho tem como objetivo avaliar o desempenho de quatro variantes de AGs, incluindo o operador de *DeleteCross* e estratégias de **clusterização** do grafo, aplicadas a instâncias de diferentes tamanhos da *TSPLIB*. Utilizando como métricas a eficiência (tempo de execução) e a eficácia (qualidade da solução final) dessas abordagens.

2 Materiais e métodos

Nesta seção estão descritos a metodologia de experimentação adotada, as variantes do Algoritmos Genéticos (AGs) implementados e o ambiente computacional dos testes.

2.1 Ambiente Computacional

Os experimentos foram executados em um computador pessoal com as seguintes configurações:

- **Processador (CPU):** Intel Core i5-13600KF (14 Núcleos/20 Threads, até 5.1 GHz).
- **Memória RAM:** 32GB DDR5 6000 MHz.
- **Sistema Operacional:** Ubuntu 24.04.2 LTS.
- **Linguagem de Programação:** Python 3.12.3.

2.2 Algoritmos Genéticos Implementados

O trabalho comparou quatro variantes do Algoritmo Genético (AG) aplicadas ao Problema do Caixeiro Viajante (TSP), sendo o objetivo principal a minimização da distância total do ciclo (função objetivo).

2.2.1 Parâmetros AG

As configurações dos parâmetros foram mantidas constantes para todos os algoritmos, sendo:

- **Número de Gerações ($n_{geracoes}$):** 500
- **Tamanho da População (tam_{pop}):** 100
- **Taxa de Elitismo ($elit_{pct}$):** 2.5%
- **Método de Seleção:** Seleção por Torneio ($k = 3$)
- **Operador de Cruzamento (Crossover):** Operador de Ordem (OX)
- **Número de Execuções por Instância:** 20

2.2.2 AG's Implementados

Os quatro algoritmos implementados no código, são:

1. **AG Original:** Utiliza o operador de **Mutação de Dois Pontos (Swap Mutation)**, onde dois pontos aleatórios do Indivíduo são trocados.
2. **AG 1DeleteCross:** Utiliza o operador **DeleteCross**, que remove um cruzamento promovendo pequenas perturbações com alto ganho.
3. **AG 1DC-Solucao-Cluster2x2:** Utiliza a estratégia de **Decomposição por Cluster** ($N = 2$). O grafo é dividido em quatro grupos, um AG 1DeleteCross é executado em cada subproblema (cluster), e as melhores rotas parciais são concatenadas para formar a solução final.
4. **AG 1DC-AG-Cluster2x2:** Aplica a decomposição do problema Cluster ($N = 2 \times 2$). Após a execução de um AG-1DeleteCross em cada cluster, as populações finais de cada grupo são agrupadas em uma nova população e é executado um segundo AG 1DeleteCross sobre o problema completo.

2.3 Instâncias de Testes

A avaliação dos algoritmos foi realizada utilizando três instâncias do **TSPLIB** (Traveling Salesman Problem Library), com diferentes tamanhos:

- **berlin52:** Instância de 52 cidades.
- **ch150:** Instância de 150 cidades.
- **rd400:** Instância de 400 cidades.

2.4 Métricas de Desempenho

O desempenho de cada algoritmo foi mensurado por três métricas, a partir das 20 execuções:

- **Fitness Inicial Médio (*fit_inicial*):** O valor médio da melhor solução encontrada na população inicial.
- **Fitness Melhor Médio (*fit_melhor*):** O valor médio da melhor solução encontrada ao final das 500 gerações.
- **Tempo Médio (*tempo_execucao*):** O tempo médio de execução em segundos para todo o processo do AG.
- **Ganho Relativo Médio (*ganho_relativo*):** O percentual de melhoria entre o Fitness Inicial e o Fitness Final, calculado por:

$$\text{Ganho Relativo} = \frac{\text{Fitness Inicial} - \text{Fitness Final}}{\text{Fitness Inicial}} \times 100\%$$

2.5 Clusterização

A clusterização permite tratar subgrupos de pontos de forma independente, aplicando algoritmos de otimização local em cada cluster antes de combinar as soluções parciais em uma solução global. A [Figura 1](#) mostra a distribuição dos clusters para uma divisão em $n \times n$ ($n=2$), onde cada cor representa um grupo distinto.

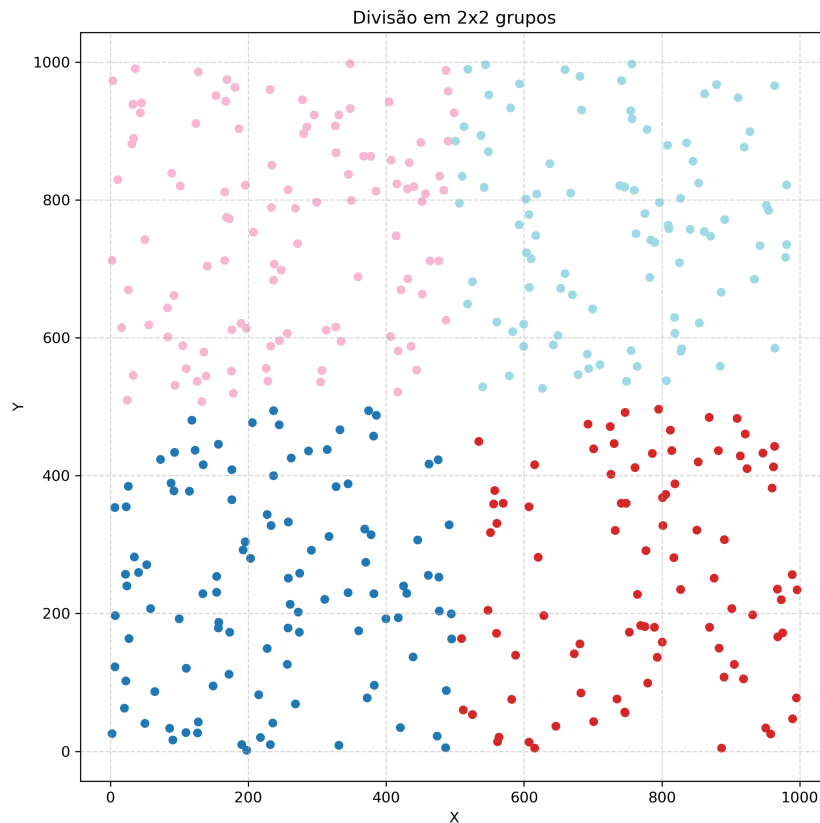


Figura 1: Divisão de pontos rd400

2.6 Particularidades de implementação

- **Operador OX** - O operador sorteia toda vez que é chamado um numero entre 0.15 e 0.40 para determinar o ponto de corte, para segunda metade (ponto final) é realizado a diferença proporcional, conforme ilustra a [Figura 2](#).

```
def operador_ox(pai1, pai2):
    tamanho = len(pai1)
    ponto_rand = random.uniform(0.15, 0.4)
    ponto_inicio = int(tamanho * ponto_rand)
    ponto_fim = int(tamanho * (1 - ponto_rand))

    centro_pai1 = pai1[ponto_inicio:ponto_fim]
    centro_pai2 = pai2[ponto_inicio:ponto_fim]

    lista1 = pai1[ponto_fim:] + pai1[:ponto_fim]
    lista2 = pai2[ponto_fim:] + pai2[:ponto_fim]
    lista1 = [x for x in lista1 if x not in centro_pai2]
    lista2 = [x for x in lista2 if x not in centro_pai1]
    filho1 = lista1[ponto_inicio:] + centro_pai2 + lista1[:ponto_inicio]
    filho2 = lista2[ponto_inicio:] + centro_pai1 + lista2[:ponto_inicio]
    return filho1, filho2
```

Figura 2: Trecho de código da função Operador OX

- **Função objetiva** — O cálculo do *fitness* pela função objetiva é realizado sempre que necessário, em vez de consultar uma matriz de adjacência pré-calculada. Assim, sendo adequado para instâncias menores, mas penalizando o desempenho em instâncias maiores.
- **Operador *delete-cross*** — O operador é aplicado n vezes por geração, onde n corresponde a aproximadamente 5% do tamanho da instância (taxa de aplicação do *delete-cross*). Além disso, ele é sempre aplicado no início da solução.
- **Junção de clusters** — Os grupos são combinados em uma solução global utilizando o ponto mais próximo entre clusters consecutivos. Para cada par de clusters, é identificado o ponto em um cluster que está mais próximo de um ponto do cluster seguinte, e a sequência de pontos é reorganizada para minimizar o salto entre eles.

3 Resultados

3.1 Berlin52

A Figura 3 apresenta os tempos de execução dos algoritmos na instância berlin52. Observa-se que o algoritmo 1DC-AG-Cluster2x2 obteve o maior tempo em todas, possuindo também o maior desvio-padrão ($\sigma = 2.52$). O 1DeleteCross e 1DC-Solucao-Cluster2x2 tiveram tempos parecidos.

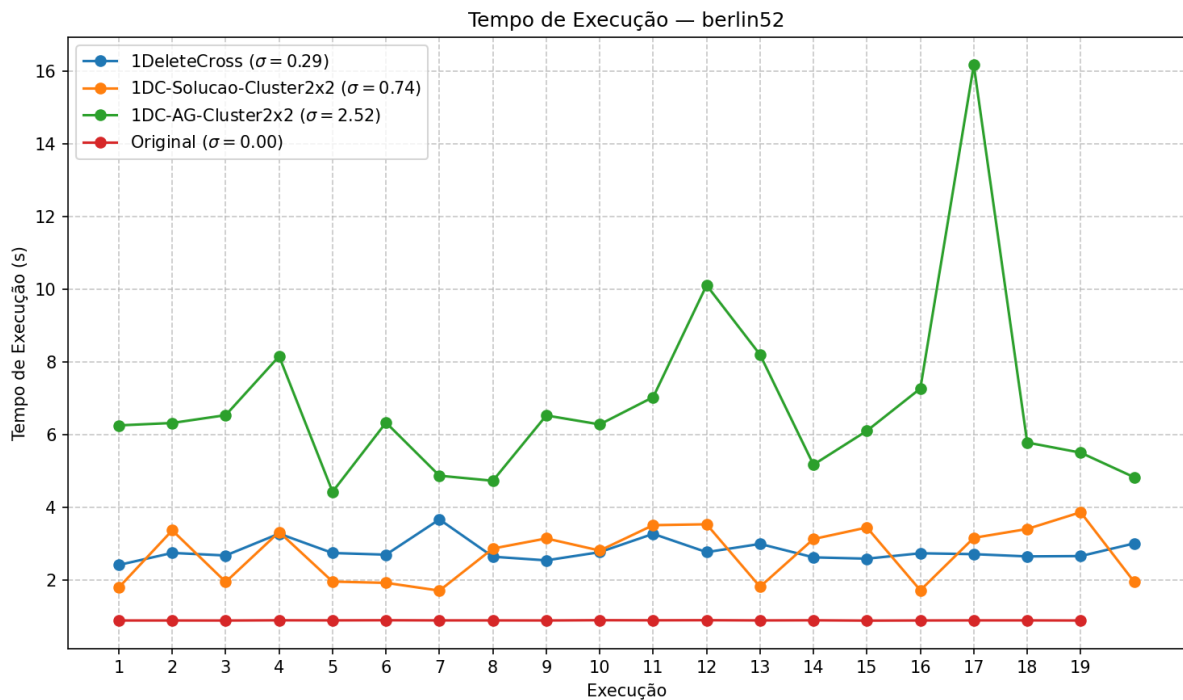


Figura 3: Tempos de execução para o conjunto de dados Berlin52.

A Figura 4 apresenta o ganho relativo de cada algoritmo, destaca-se que neste caso, o 3 algoritmos que utilizam a mutação por *delete-cross* obtiveram um ganho consideravelmente parecidos, enquanto o AG Original teve o pior ganho, evidenciando que a abordagem de mutação por dois pontos não é ideal para problemas TSP.

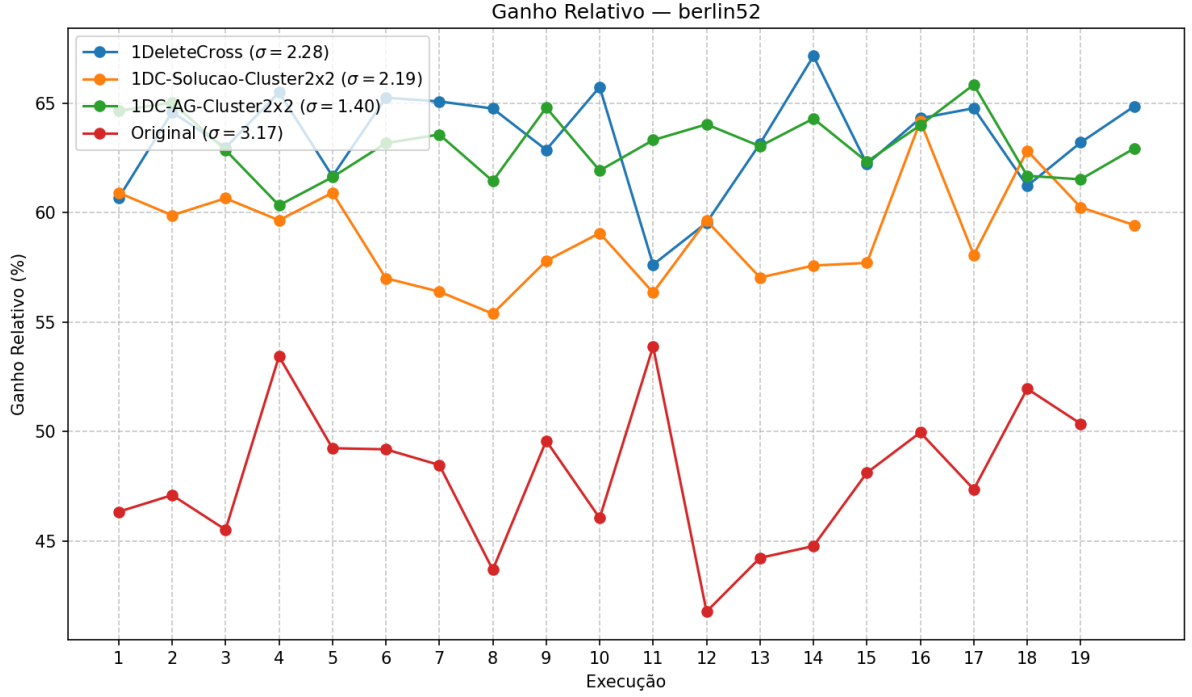


Figura 4: Ganho relativo obtido pelos algoritmos no conjunto Berlin52.

3.2 CH150

A Figura 5 ilustra os tempos de execução para a instância de 150 cidades, observa-se que a clusterização melhora significativamente o desempenho dos algoritmos. O algoritmo 1DeleteCross e 1DC-Solucao-Cluster-2x2 obtiveram tempos parecidos, porém o segundo com um desvio padrão maior ($\sigma = 3.74$).

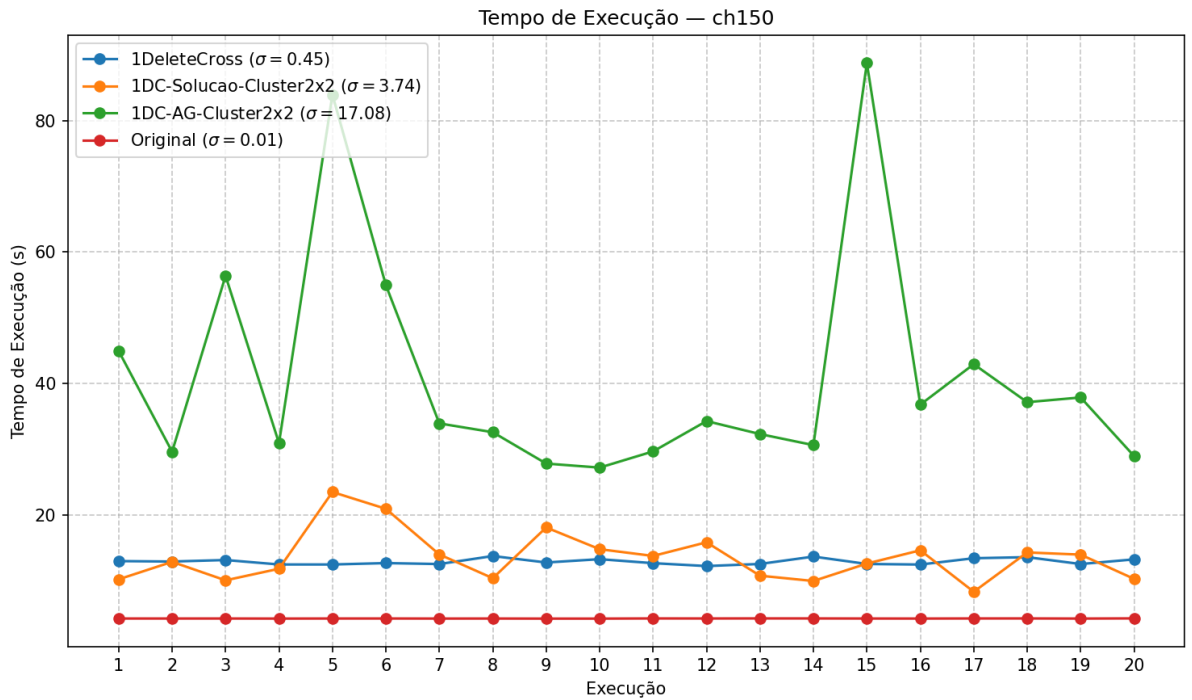


Figura 5: Tempos de execução para o conjunto de dados CH150.

O ganho relativo referente a instância é apresentado na Figura 6. O AG Original apresentou baixo ganho relativo (41.20%) apesar do menor tempo de execução (4.25 s), evidenciando que a mutação simples não é adequada para instâncias maiores. O 1DeleteCross melhorou consideravelmente o ganho relativo (60.39%) em relação ao Original, mas ainda ficou atrás das soluções com clusterização que obtiveram desempenhos consideravelmente parecidos.

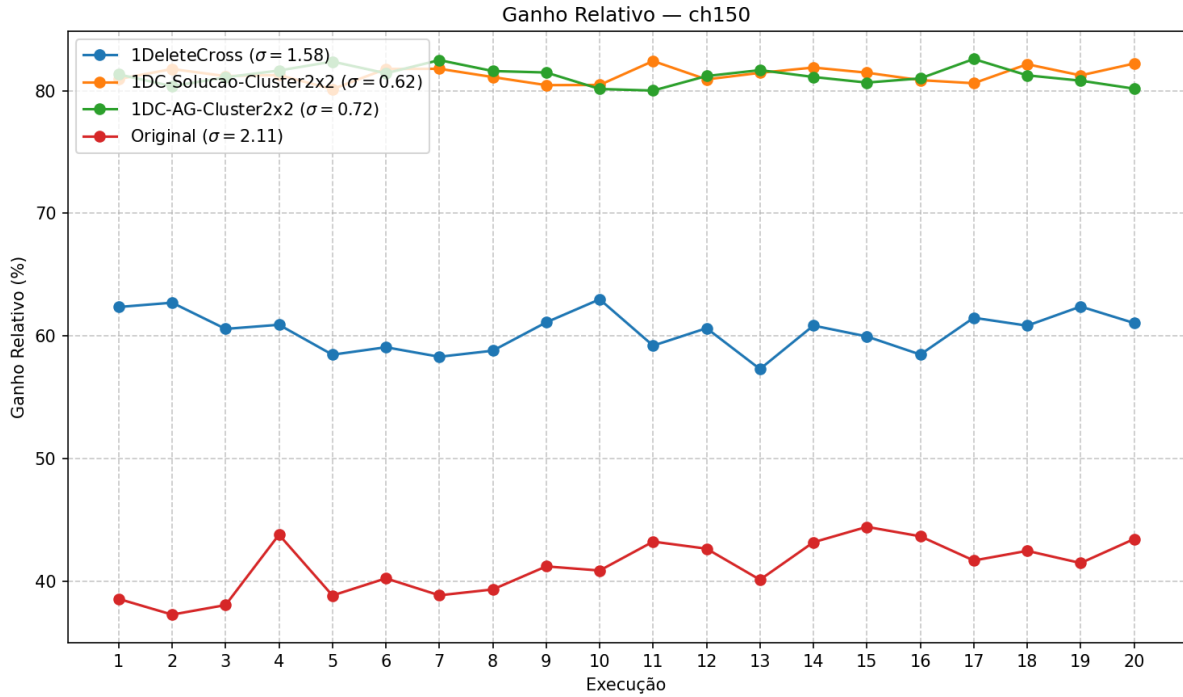


Figura 6: Ganho relativo obtido pelos algoritmos no conjunto CH150.

3.3 RD400

A Figura 7 apresenta o tempo de execucao na maior instância de teste. Nota-se que o tempo do 1DC-Solucao-Cluster-2x2 foi menor que o algoritmo sem a clusterizaração 1DeleteCross, se aproximando do tempo do AG Original.

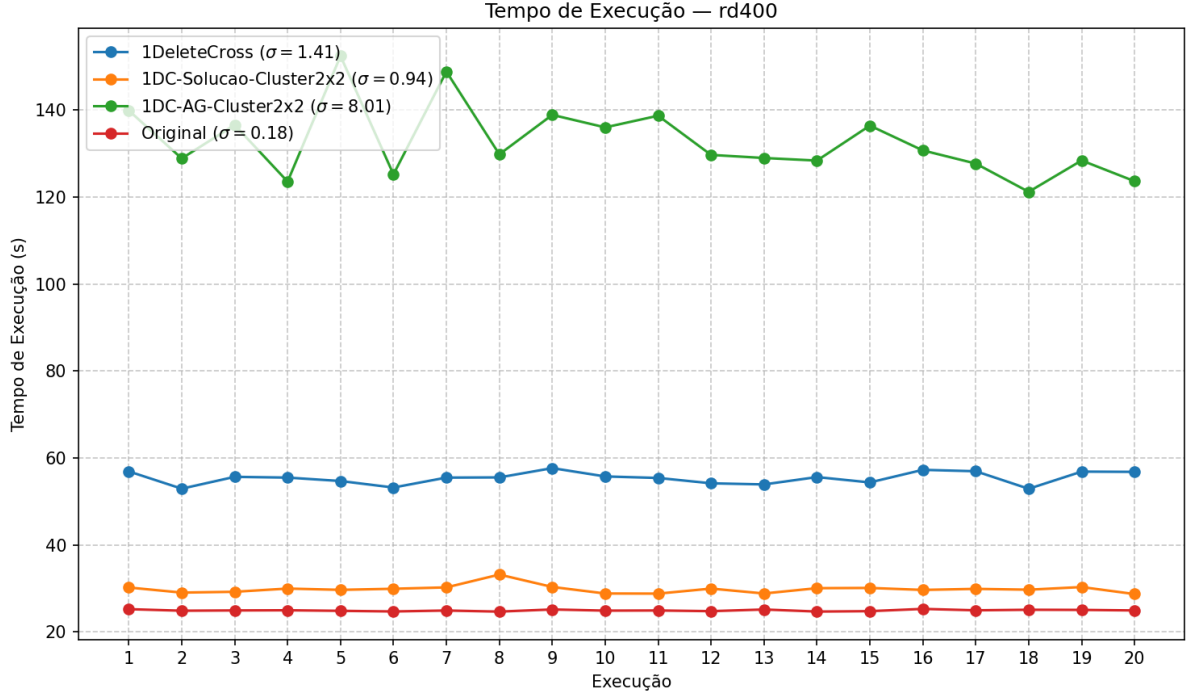


Figura 7: Tempos de execução para o conjunto de dados RD400.

A Figura 8 apresenta o ganho relativo dos algoritmos. O 1DC-Solucao-Cluster-2x2 e 1DC-AG-Cluster-2x2 apresentam o mesmo ganho com desvio-padrão consideravelmente parecidos ($\sigma = 0.60$ e $\sigma = 0.69$, respectivamente).

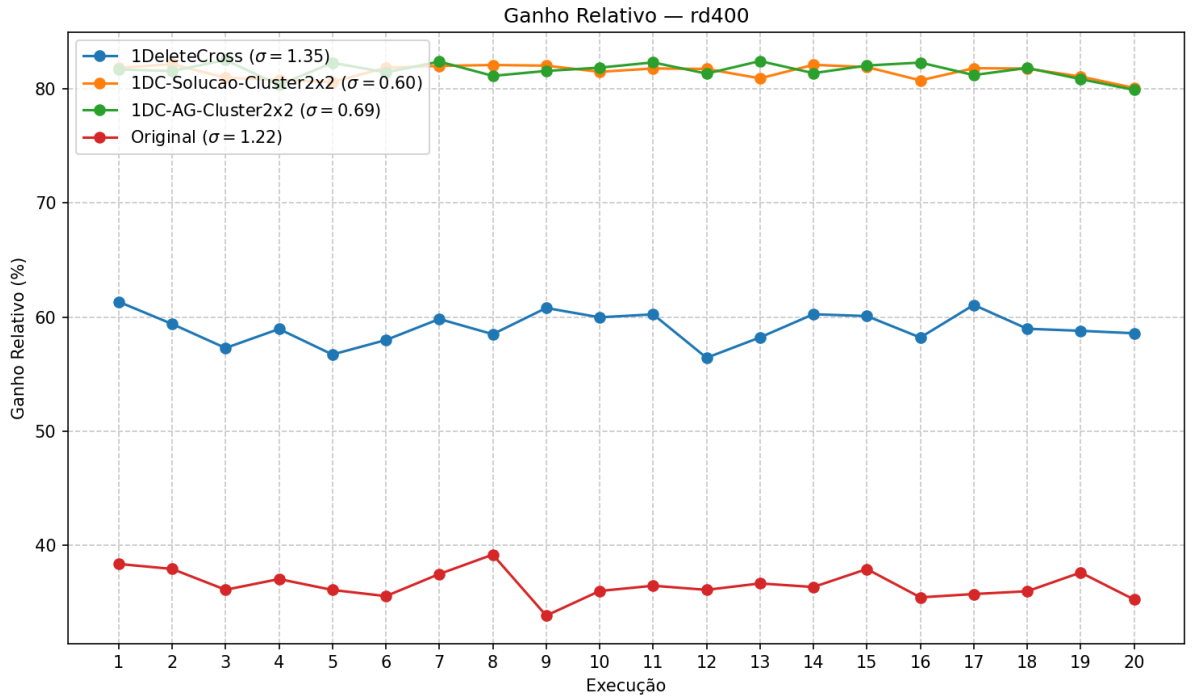


Figura 8: Ganho relativo obtido pelos algoritmos no conjunto RD400.

3.4 Resultados gerais

A Tabela 1 apresenta os resultados gerais obtidos ao fim das 20 execuções.

- O **Algoritmo Original** foi consistentemente o pior em todas as instâncias em termos de qualidade da solução (**fit_melhor** e **ganho_relativo**), apesar do menor tempo de execução.
- O **1DeleteCross** foi o melhor para a menor instância (**berlin52**), com o maior ganho relativo (63.36%).
- Para instâncias maiores (**ch150** e **rd400**), as abordagens com **clusterização** apresentaram os maiores ganhos relativos (acima de 81%).
- Os ganhos de **1DC-Solucao-Cluster2x2** e **1DC-AG-Cluster2x2** são muito próximos, porém:
 - **1DC-Solucao-Cluster2x2** foi significativamente mais rápido para **ch150** (13.56s vs 41.09s).
 - **1DC-AG-Cluster2x2** obteve a melhor solução para **rd400**.

Grafo	Algoritmo	Fit Inicial Médio	Fit Melhor Médio	T. Médio (s)	Ganho Rel. Médio (%)
berlin52	Original	25874.49	13453.63	0.89	47.95
berlin52	1DeleteCross	25920.39	9483.37	2.81	63.36
berlin52	1DC-Solucao-Cluster2x2	26040.31	10662.71	2.72	59.04
berlin52	1DC-AG-Cluster2x2	25875.96	9540.23	6.83	63.12
ch150	Original	49142.68	28886.39	4.25	41.20
ch150	1DeleteCross	49201.46	19481.41	12.90	60.39
ch150	1DC-Solucao-Cluster2x2	49167.15	9194.53	13.56	81.30
ch150	1DC-AG-Cluster2x2	49237.51	9248.64	41.09	81.21
rd400	Original	200077.12	126943.52	24.93	36.55
rd400	1DeleteCross	200834.91	82159.24	55.36	59.09
rd400	1DC-Solucao-Cluster2x2	200579.27	37108.09	29.82	81.50
rd400	1DC-AG-Cluster2x2	199967.90	36727.31	132.68	81.63

Tabela 1: Resultados médios obtidos para cada grafo.

Observa-se na Figura 9 que o **1DeleteCross** apresenta a melhor relação ganho/tempo para a instância **berlin52**. Para instâncias maiores (**ch150** e **rd400**), as abordagens com clusterização (**1DC-Solucao-Cluster2x2** e **1DC-AG-Cluster2x2**) apresentam ganhos relativos maiores, porém a eficiência em termos de tempo se torna mais sensível. O **1DC-Solucao-Cluster2x2** oferece um bom desempenho entre ganho e tempo, enquanto o **1DC-AG-Cluster2x2** alcança as melhores soluções finais, mas com custo de tempo significativamente maior.

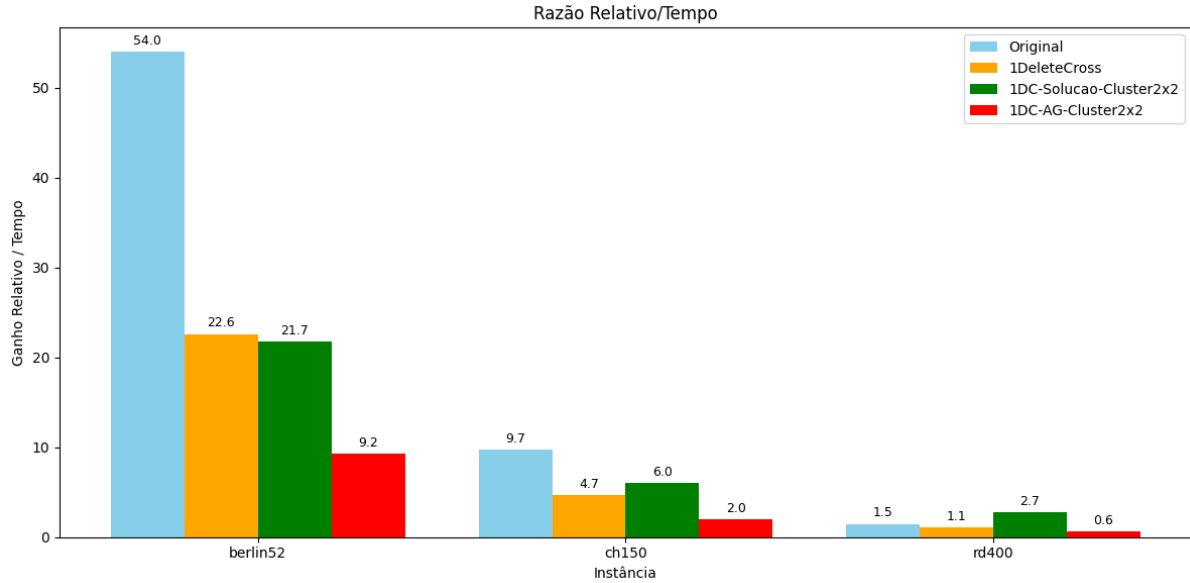


Figura 9: Razão entre Ganho Relativo e Tempo de execução.

4 Conclusões

O AG Original apresentou o menor tempo de execução entre as abordagens, porém forneceu as piores soluções finais e o menor ganho relativo. Apesar de ser computacionalmente barato, o algoritmo realiza uma exploração limitada do espaço de busca, restringindo seu potencial ao longo das gerações.

O 1DeleteCross demonstrou desempenho superior, especialmente para instâncias pequenas, como a **berlin52**. Nesses casos, o algoritmo alcançou soluções finais de boa qualidade com um tempo de execução médio. Porém, seu desempenho relativo diminui à medida que o tamanho da instância aumenta, não escalando para problemas maiores.

Por fim, as variantes com **clusterização** (1DC-Solucao-Cluster2x2 e 1DC-AG-Cluster2x2) apresentaram os maiores tempos de execução, porém alcançaram as melhores soluções finais e os maiores ganhos relativos em todas as instâncias maiores. Reforçando que a decomposição em subproblemas permite uma melhor exploração do espaço de busca. Portanto, os métodos baseados em clusterização mostraram-se os mais eficazes para instâncias de maior porte.

Referências

M. Arenales, R. Morabito, V. Armentano, and H. Yanasse. *Pesquisa Operacional: Para cursos de engenharia*. Elsevier Brasil, 2015. ISBN 9788535281835.